LAPORAN ETS

PRAKTIKUM IMCLab pada Modul Satu hingga Empat

Mata Kuliah

ROBOTIKA-A

Dosen Pengampu

Dr. Basuki Rahmat, S.Si. MT

Disusun Oleh:

Cantika Berliana Rahmasari 23081010245

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

I. Pendahuluan

iMCLab adalah kit praktikum kontrol berbasis mikrokontroler ESP32. Praktikum dibagi menjadi empat modul yang disusun bertahap: dimulai dari antarmuka serial paling dasar, otomasi pengambilan data dengan Python, pemantauan dan kendali jarak jauh lewat platform IoT Blynk, sampai pembaruan firmware tanpa kabel (*Over-The-Air*, OTA). Laporan ini merangkum tujuan, rancangan program, cara kerja, serta hasil dan analisis dari keempat modul berdasarkan berkas sketch, skrip, dan data yang dihasilkan.

I.1 Tujuan

- Memahami pemrograman ESP32 untuk membaca sensor dan menggerakkan aktuator (PWM) melalui antarmuka serial.
- Mengotomasi pengujian step PWM, pencatatan data ke CSV, dan visualisasi respons motor dengan Python.
- Membangun kendali dan pemantauan motor (PWM, arah, RPM) melalui aplikasi Blynk IoT.
- Menerapkan pembaruan firmware jarak jauh (OTA) beserta penomoran versi firmware.

I.2 Ringkasan Empat Modul

Modul	Berkas	Inti pekerjaan	Antarmuka
1 — Basic	1_imclab_basic.ino	Baca suhu T1/T2, kontrol heater Q1/Q2 dan LED lewat perintah teks	Serial Monitor 115200 baud
2 — Python	imclab_modul2.py	Uji step PWM otomatis, catat data CSV, buat grafik	Serial dari PC (pyserial)
3 — Blynk	imclab_modul3_blynk.ino	Kendali PWM/arah/LED dan kirim RPM ke dashboard	WiFi + Blynk IoT
4 — OTA	imclab_modul4_final.ino	Modul 3 + versi firmware, batas PWM, pembaruan OTA	WiFi + Blynk OTA + ArduinoOTA

2. Perangkat dan Konfigurasi

Perangkat keras utama adalah papan ESP32 yang dihubungkan ke driver motor DC (jalur *enable* PWM dan dua pin arah), sensor putaran berupa keluaran pulsa dari cakram/baling-baling, dan LED biru *onboard*. Perangkat lunak yang dipakai: Arduino IDE (board *ESP32 Dev Module*), pustaka Blynk (core 3.x), ArduinoOTA, serta Python dengan paket p y s e r i a l dan m a t p l o t l i b .

Fungsi	Modul 1 (GPIO)	Modul 3 dan 4 (GPIO)
PWM aktuator	Q1 = 32, Q2 = 33 (heater)	EN = 12 (kecepatan motor)
Arah motor	—	IN1 = 27, IN2 = 26
Sensor	T1 = 34, T2 = 35 (ADC)	Encoder/pulsa = 13 (interrupt RISING)
LED	LED1 = 26 (PWM), LED2 = 27	LED biru onboard = 2
Frekuensi/resolusi PWM	5 kHz / 8 bit	30 kHz / 8 bit

3. Modul 1 — iMCLab Basic (Serial)

3.1 Tujuan dan Cara Kerja

Sketch menerima perintah teks lewat Serial Monitor (115200 baud, akhiran *Newline*) dan menjawab dengan satu baris teks. Protokolnya meniru TCLab sehingga program Python dapat berkomunikasi dengan papan yang sama. Setiap iterasi `loop()` membaca suhu T1 dan T2, memeriksa batas aman, lalu memproses perintah bila ada data serial.

3.2 Perintah yang Didukung

Perintah	Fungsi	Jawaban
VER	Menampilkan versi firmware	iMCLab Basic 1.0
T1 | T2	Membaca suhu sensor 1 atau 2 (°C)	Angka, 2 desimal
Q1 n | Q2 n	Mengatur heater 1 atau 2 sebesar n persen (0–100)	Nilai n (dibatasi 0–100)
LED n	Mengatur kecerahan LED (0–100 persen)	Nilai n (dibatasi 0–100)
X	Mematikan semua heater	Stop
lainnya	Perintah tidak valid	Perintah tidak dikenal

3.3 Penjelasan Program

- **Pembacaan suhu** (`readTemp`): 20 sampel ADC 12-bit dirata-ratakan untuk menekan derau, diubah ke tegangan (0–3,3 V), lalu dikali 100 dengan asumsi sensor LM35 (10 mV/°C). Faktor ini perlu disesuaikan bila sensor yang dipakai berbeda.
- **PWM**: kanal LEDC 5 kHz, 8 bit. Fungsi `setPct` membatasi nilai 0–100 lalu memetakan ke 0–255.
- **Pengaman suhu**: bila T1 atau T2 melebihi `TEMP_MAX` = 80 °C, `allOff()` mematikan kedua heater secara otomatis.
- **Parser perintah**: baris dibaca sampai *newline*, di-*trim*, diubah ke huruf besar, lalu dicocokkan dengan `if / else`.

```
float readTemp
  (int pin) {lon
  g sum = 0;
  for (int i = 0; i < 20; i++) sum
  += analogRead(pin); float volt
  = (sum / 20.0) / 4095.0 * 3.3;
  return volt * 100.0;    // LM35: 10 mV per derajat C
}
void loop() {
  float t1 = readTemp(pinT1), t2 = readTemp(pinT2);
  if (t1 > TEMP_MAX || t2 > TEMP_MAX) allOff(); // pengaman
  ...
}
```

3.4 Hasil dan Pembahasan

Modul 1 menjadi fondasi komunikasi: semua perintah bersifat *request–response* satu baris sehingga mudah diuji manual dan mudah diotomasi dari komputer. Pengaman suhu pada setiap iterasi loop memastikan aktuator mati bila terjadi kondisi berlebih. Keterbatasan: ADC ESP32 tidak sepenuhnya linear, sehingga akurasi suhu dapat ditingkatkan dengan kalibras terhadap termometer acuan.

4. Modul 2 — Pemrograman iMCLab dengan Python

4.1 Tujuan dan Cara Kerja

Skrip `imclab_modul2.py` mengirim perintah ke papan lewat port serial, menjalankan *uji step PWM* sesuai profil yang ditentukan, mencatat waktu–PWM–RPM ke CSV, lalu menggambar grafik respons. Port (`COM3`) dan baud rate (115200) diatur di awal skrip; Serial Monitor Arduino harus ditutup karena satu port tidak bisa dipakai dua program.

4.2 Alur Program

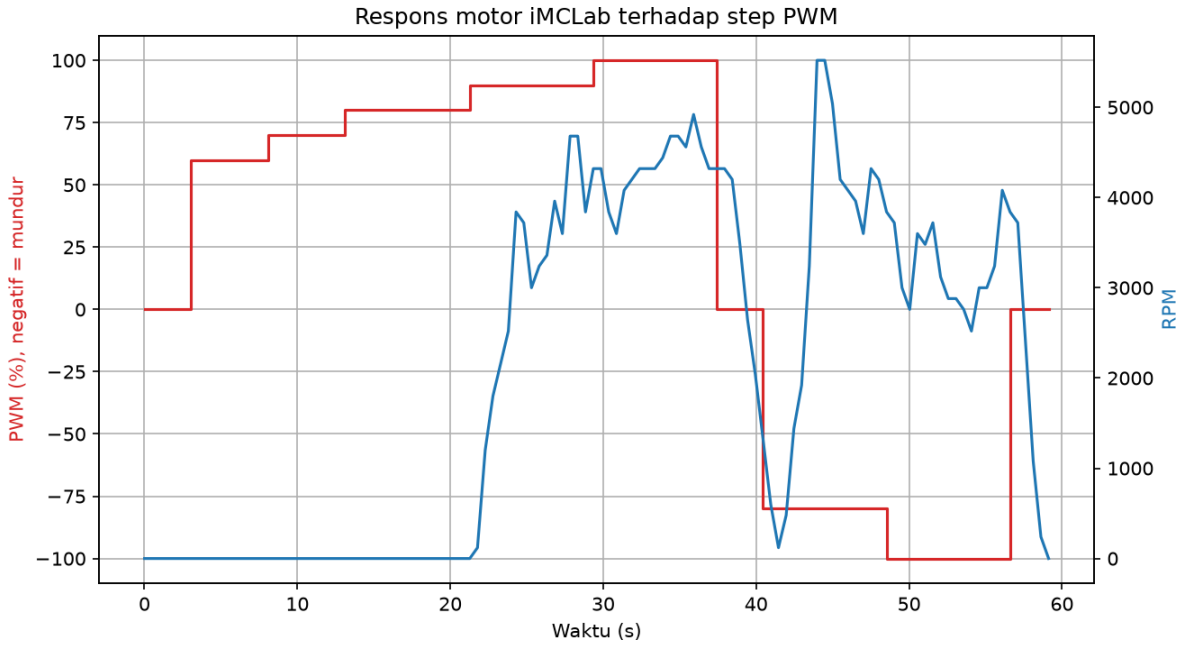
- **buka_port()**: membuka port dengan DTR dan RTS dimatikan agar ESP32 tidak ter-*reset* saat port dibuka, lalu jeda 1 detik.
- **kirim()**: membersihkan buffer masukan, menulis perintah + newline, dan membaca satu baris jawaban.
- **Handshake**: mengirim `VER` hingga 5 kali; jawaban harus diawali "iMCLab", jika tidak program berhenti dengan pesan pemecahan masalah.
- **Uji step**: untuk tiap elemen profil dikirim `DIR` dan `PWM`, lalu RPM dibaca berulang setiap 0,5 detik selama durasi segmen.
- **Keamanan**: blok `finally` selalu mengirim `X` (stop) dan menutup port, termasuk bila dihentikan dengan Ctrl+C.
- **Keluaran**: `data_imclab.csv` (kolom waktu_s, pwm_bertanda_persen, rpm) dan `grafik_imclab.png` (150 dpi). PWM mundur dicatat bertanda negatif.

4.3 Profil Uji

No	PWM (%)	Arah	Durasi (s)	Tujuan
1	0	maju	3	Kondisi awal (diam)
2–3	60, 70	maju	5 + 5	Menunjukkan zona mati motor
4–6	80, 90, 100	maju	8 + 8 + 8	Mencari ambang start dan respons beban penuh
7	0	maju	3	Respons pelepasan (motor meluruh)
8–9	80, 100	mundur	8 + 8	Uji putar balik
10	0	mundur	3	Berhenti

4.4 Hasil Pengukuran

Pengujian menghasilkan 118 sampel selama ± 59 detik (selang rata-rata 0,505 detik). Gambar 1 menampilkan PWM (merah, sumbu kiri) dan RPM (biru, sumbu kanan).



Gambar 1. Respons motor iMCLab terhadap step PWM (grafik_imclab.png).

Segmen (waktu)	PWM	RPM rata-rata	RPM maks	Keterangan
0–2,5 s	0 %	0	0	Diam
3,0–7,6 s	+60 %	0	0	Zona mati
8,1–12,6 s	+70 %	0	0	Zona mati
13,1–20,8 s	+80 %	0	0	Masih zona mati
21,3–28,8 s	+90 %	2.858	4.680	Motor mulai berputar ± 0,5 s setelah PWM naik; rata-rata memuat fase start
29,4–36,9 s	+100 %	4.342	4.920	Paling stabil (rentang 3.600–4.920 RPM)
37,4–39,9 s	0 % (maju)	3.500	4.320	Meluruh bebas, dari 4.320 ke 2.040 dalam ± 2,5 s
40,5–48,0 s	−80 %	3.098	5.520	Turun ke ± 120 RPM (pembalikan arah), lalu naik dan berfluktuasi
48,5–56,1 s	−100 %	3.225	4.080	Rentang 2.520–4.080 RPM
56,6–59,1 s	0 % (mundur)	1.880	3.840	Meluruh sampai 0

4.5 Analisis

- **Zona mati.** Pada PWM 60–80 % motor tidak berputar sama sekali (RPM = 0); putaran baru terdeteksi pada PWM 90 %, sekitar 21,8 detik dari awal uji. Ambang start motor berada di antara 80 % dan 90 % duty cycle, sehingga PWM di bawah itu tidak menghasilkan gerak.

- **Hubungan PWM–RPM.** Di atas ambang, kenaikan PWM dari 90 % ke 100 % menaikkan RPM ke sekitar 4.300 RPM rata-rata. Respons bersifat sangat tidak linear karena adanya zona mati.
- **Respons pelepasan.** Saat PWM diturunkan ke 0 % motor tidak berhenti seketika, melainkan meluruh sesuai inersia rotor dan beban (± 2,5 detik dari 4.320 ke 2.040 RPM).

- **Asimetri saat mundur.** Pada −80 % motor sudah berputar sehingga RPM tinggi, padahal +80 % dari keadaan diam tidak menggerakkan motor. Ini konsisten dengan gesekan statis yang lebih besar daripada gesekan dinamis (histeresis). Lonjakan sampai 5.520 RPM kemungkinan dipengaruhi derau atau kesalahan hitung pulsa, bukan kecepatan sebenarnya.
- **Resolusi pengukuran.** Nilai RPM tampak berkelipatan 120 (misalnya 3.600, 4.680, 4.920). Ini sesuai dengan 1 pulsa per putaran pada jendela hitung ± 0,5 detik, sehingga satu pulsa setara 120 RPM; fluktuasi ± 120–240 RPM pada kecepatan tinggi adalah efek kuantisasi.
- **Arah tidak terbaca.** Sensor hanya menghitung pulsa, sehingga RPM selalu positif; arah putaran hanya diketahui dari perintah yang dikirim.

4.6 Tujuan dan Cara Kerja

ESP32 terhubung ke WiFi 2.4 GHz dan ke server Blynk IoT. Dashboard di aplikasi Blynk mengirim nilai slider PWM, saklar arah, dan saklar LED; sebaliknya, firmware mengirim RPM ke dashboard setiap 1 detik. Pustaka yang dipakai adalah Blynk (core 3.x) pada board ESP32 Dev Module.

4.7 Datastream

Pin virtual	Nama	Tipe / rentang	Widget	Arah data
V0	RPM	Double, 0–5000	Gauge / Chart	ESP32 → Blynk
V1	PWM	Integer, 0–100	Slider	Blynk → ESP32
V2	Arah	Integer, 0–1	Switch (1 = maju, 0 = mundur)	Blynk → ESP32
V3	LED	Integer, 0–1	Switch (LED biru onboard)	Blynk → ESP32

4.8 Penjelasan Program

- **Kredensial**: *Template ID*, *Template Name*, dan *Auth Token* diambil dari Blynk Console; SSID dan kata sandi WiFi ditulis di sketch (nilai sebenarnya tidak dicantumkan dalam laporan ini).
- **Pencacah pulsa**: o n P u l s e ( ) adalah ISR (I R A M _ A T T R) pada tepi naik GPIO 13 yang menambah variabel v o l a t i l e p u l s e s . Filter derau M I N _ P U L S E _ U S pada Modul 3 bernilai 0, artinya tidak ada pulsa yang diabaikan.
- **Perhitungan RPM**: k i r i m R P M ( ) dijalankan B l y n k T i m e r tiap 1000 ms. Pulsa dibaca dan di-nol-kan di dalam bagian kritis (n o I n t e r r u p t s), lalu RPM = (pulsa / PULSES_PER_REV) × 60000 / selang_ms.
- **Penggerak motor**: a p p l y M o t o r ( ) mengatur IN1/IN2 sesuai arah dan menulis duty cycle ke pin EN (l e d c A t t a c h , sintaks core 3.x). Konstanta P W M _ S T A R T (di sini 0) dapat diisi 80 agar slider 1–100 dipetakan ke 80–100 % sehingga zona mati terlewati.
- **Sinkronisasi**: B L Y N K _ C O N N E C T E D ( ) memanggil s y n c V i r t u a l agar nilai terakhir dari server diterapkan saat tersambung ulang.

```
void apply
  Motor() {
  int efekti
  f = 0;
  if (pwmPct > 0) efektif = PWM_START + pwmPc
  t * (100 - PWM_START) / 100;digitalWrite(PIN
  _IN1, dirNow ? LOW : HIGH);
  digitalWrite(PIN_IN2, dirNo
  w ? HIGH : LOW);ledcWrite
  (PIN_EN, efektif * 255 / 10
  0);
}
BLYNK_WRITE(V1) { pwmPct = constrain(param.asInt(), 0, 100); applyMo
tor(); }
```

4.9 Hasil dan Pembahasan

Modul 3 memindahkan kendali dari kabel serial ke jaringan: motor dapat dikontrol dan diamati dari telepon pintar. Hubungan dengan hasil Modul 2 sangat jelas — karena zona mati mencapai ± 80 %, slider yang bekerja pada 0–100 % membuat sebagian besar rentang slider tidak berpengaruh. Opsi PWM_START disediakan tepat untuk mengatasi hal ini. Perhitungan RPM per 1 detik dengan 1 pulsa/putaran memberi resolusi 60 RPM per pulsa, dua kali lebih halus dibanding jendela 0,5 detik pada Modul 2.

5. Modul 4 — Pemrograman OTA berbasis Blynk

5.1 Tujuan dan Cara Kerja

Modul 4 menambahkan kemampuan memperbarui firmware tanpa kabel USB. Ada dua jalur: **Blynk.Air** (OTA lewat cloud Blynk, memakai BLYNK_FIRMWARE_TYPE dan BLYNK_FIRMWARE_VERSION) dan **ArduinoOTA** sebagai cadangan lewat WiFi lokal dengan nama host imclab. Nomor versi dikirim ke datastream V5 sehingga dashboard menampilkan versi firmware yang sedang berjalan.

5.2 Perubahan terhadap Modul 3

Aspek	Modul 3	Modul 4
Versi firmware	Tidak ada	BLYNK_FIRMWARE_VERSION = "1.0.1" (dinaikkan tiap update), ditampilkan di V5 dan Serial
OTA	Tidak ada	Blynk.Air (ditandai type + version) dan ArduinoOTA (host “imclab”)
Batas PWM	0–100 %, opsi PWM_START	PWM_MAX = 90; permintaan di atas batas dipotong dan slider dikembalikan

		lewat virtualWrite
Filter derau pulsa	MIN_PULSE_US = 0 (nonaktif)	MIN_PULSE_US = 200 µs (pulsa lebih rapat diabaikan)
Keselamatan saat update	—	ArduinoOTA.onStart() menghentikan motor (pwmPct = 0)
Log serial	pulsa, RPM, PWM, arah	Ditambah awalan versi (v1.0.1 ...)
Loop utama	Blynk.run(), timer.run()	Ditambah ArduinoOTA.handle()

5.3 Prosedur Pembuktian Update OTA

Unggah versi awal lewat kabel; dashboard menampilkan versi pada V5 dan batas PWM bekerja sesuai nilai awal.

- Ubah `PWM_MAX` (mis. dari 90 ke nilai lain) dan naikkan `BLYNK_FIRMWARE_VERSION` (1.0.1 → 1.0.2).
- Ekspor biner terkompilasi dari Arduino IDE, lalu kirim sebagai *firmware update* melalui Blynk (atau lewat port jaringan ArduinoOTA).
- Setelah papan restart, periksa label versi di V5 dan uji bahwa slider kini terbatas pada nilai `PWM_MAX` yang baru.

5.4 Hasil dan Pembahasan

Perubahan P W M _ M A X berfungsi sebagai bukti yang mudah diamati bahwa firmware benar-benar tergantikan, sementara penomoran versi mencegah kebingungan mengenai versi mana yang sedang berjalan. Penghentian motor saat OTA dimulai merupakan praktik keselamatan penting karena program berhenti merespons selama penulisan flash.

6. Perbandingan Antarmodul

Aspek	Modul 1	Modul 2	Modul 3	Modul 4
Media kendali	Serial Monitor	Skrip Python (serial)	Aplikasi Blynk (WiFi)	Aplikasi Blynk + OTA
Otomasi	Manual	Otomatis (profil step)	Manual via dashboard	Manual via dashboard
Pencatatan data	Tidak ada	CSV + grafik PNG	Grafik di dashboard Blynk	Grafik di dashboard Blynk
Jarak kendali	Kabel USB	Kabel USB	Internet	Internet + OTA
Pembaruan firmware	Kabel	Kabel	Kabel	Nirkabel (OTA)
Pengaman	Suhu maks. 80 °C	Perintah X di finally	Tidak ada batas khusus	PWM_MAX, stop saat OTA

7. Kesimpulan

- Keempat modul membentuk alur yang runtut: komunikasi serial dasar (Modul 1), otomasi dan analisis data (Modul 2), kendali jarak jauh berbasis IoT (Modul 3), dan pemeliharaan firmware nirkabel (Modul 4).
- Uji step pada Modul 2 menunjukkan zona mati motor sampai PWM 80 %, ambang start di antara 80–90 %, dan RPM ± 4.300 pada PWM 100 % dengan fluktuasi akibat kuantisasi pulsa dan derau.
- Pada Modul 3 dan 4, temuan zona mati menjadi dasar fitur pemetaan PWM (P W M _ S T A R T) dan pembatasan (P W M _ M A X).
- Penambahan filter pulsa (200 µs), pembatas PWM, penomoran versi, dan penghentian motor saat OTA membuat firmware akhir lebih andal dan aman.